

ThirdLayer Takehome

Design and build a hosted platform plus SDK that lets developers create flexible tables (tables they define with their own schemas), sync data into them from arbitrary external sources, and query that data back through an API and a client SDK.

A developer using the platform writes a project that:

1. Declares one or more tables with a typed, flexible schema.
2. Declares one or more syncs. Each fetches records from some external source and returns the rows to write into a table.
3. Deploys that project so the platform runs it: it creates and owns the tables and executes the syncs on a schedule.

The platform owns the lifecycle of the tables and the execution of the syncs. The developer only describes what to sync; the platform handles creating the tables, running the code, persisting state, and keeping the data current. Developers read and write the data through the client-side SDK.

Assumptions

Assume the platform is multi-tenant. Each user may create many tables, each with its own schema. Your design should handle this comfortably: both how schemas are stored and how the data is kept isolated and queryable per tenant.

Note that the code below is a basic reference. The SDK surface, the schema format, and the query interface are all yours to design.

Objective

1. Authoring SDK (TypeScript)

An ergonomic API a developer imports to declare a table with a flexible schema and bind syncs to it.

Support a reasonable set of field types.

Reference shape:

```
// project file: defines what to sync (illustrative, design your own surface)
import { table, sync, field } from "<your-sdk>";
```

```
const issues = table("issues", {
  primaryKey: "id",
  schema: {
    id: field.text(),
    title: field.text(),
    state: field.select(["open", "in_progress", "closed"]),
    assignee: field.text(),
    comments: field.number(),
    labels: field.multiSelect(),
    createdAt: field.datetime(),
    isUrgent: field.boolean(),
  },
});
```

Syncs must support both modes, stateful resumable pagination, and a schedule:

- replace: the sync returns the full current dataset; rows no longer present upstream are removed.
- incremental: the sync returns only changes since the last run (creates, updates, deletes).
- stateful execution: execute receives the previous run's state and returns the next state plus whether there is more to fetch. The platform persists state across runs and keeps invoking until a run completes, so a sync can paginate a large dataset across invocations.
- schedule: how often the sync runs automatically. By default it's manual.

Reference shape, a full-snapshot (replace) sync of Linear issues:

```
sync("linear-issues", {
  table: issues,
  mode: "replace",
  schedule: "15m",
  async execute(state) {
    const page = state?.page ?? 1;
    const { items, hasNextPage } = await linear.listIssues({ page });

    return {
      changes: items.map((i) => ({
        type: "upsert",
        key: i.id,
        values: {
          id: i.id,
          title: i.title,
          state: i.state,
          assignee: i.assignee?.name ?? "",

```

```

      comments: i.commentCount,
      labels: i.labels,
      createdAt: i.createdAt,
      isUrgent: i.priority === 1,
    },
  ]),
  hasMore: hasNextPage,
  nextState: hasNextPage ? { page: page + 1 } : undefined,
};
},
});

```

Reference shape, an incremental sync of GitHub pull requests using a cursor:

```

sync("github-prs", {
  table: pullRequests,
  mode: "incremental",
  schedule: "5m",
  async execute(state) {
    const since = state?.since ?? "1970-01-01T00:00:00Z";
    const { results, nextCursor, done } = await github.listPRs({ since });

    return {
      changes: results.map((pr) =>
        pr.deleted
          ? { type: "delete", key: pr.id }
          : { type: "upsert", key: pr.id, values: mapPr(pr) },
      ),
      hasMore: !done,
      nextState: { since: nextCursor },
    };
  },
});

```

2. Platform

A service that runs deployed projects:

- Manages the tables. On deploy, create them from the declared schema. On redeploy with a changed schema, migrate the existing table to match. You decide how to handle additions, removals, and type changes, and what happens to existing data. Be explicit about the trade-offs.
- Runs the syncs. Execute each sync on its schedule, drive the stateful pagination loop to completion, and apply the returned changes (upserts and deletes, deduplicated by primary key) to the table.

- Persists what it needs (synced data, schemas and metadata, per-sync state) while keeping tenants isolated.

3. Data access (REST API + client SDK)

Developers read and write synced data through the platform, not by connecting to the database themselves.

- A REST API over the tables: query rows with filters, sorts, and pagination, fetch a single row, and create, update, or delete rows.
- A client-side TypeScript SDK that wraps the REST API with an ergonomic, typed interface for those operations.

Support per-property filter conditions, compound and/or filters, multi-key sorts, and cursor pagination. You decide the exact shape.

Reference shape:

```
import { Client } from "<your-client-sdk>";

const client = new Client();

// Query rows with filters, sorts, and pagination.
const response = await client.query({
  table: "issues",
  filter: {
    and: [
      { property: "state", select: { equals: "open" } },
      { property: "isUrgent", checkbox: { equals: true } },
    ],
  },
  sorts: [{ property: "createdAt", direction: "descending" }],
  page_size: 25,
  start_cursor: cursor,
});

for (const row of response.results) {
  // ...
}
// response.has_more, response.next_cursor

// Single-row operations.
const row = await client.getRow({ table: "issues", key: "ISSUE-123" });
await client.updateRow({ table: "issues", key: "ISSUE-123", values: { state: "closed" } });
```

4. Deployment

A way to get a developer's project running on the platform: the code executes, syncs run on schedule, and data is written and queryable. It does not need to be a CLI or a remote host; running locally is fine, and a `.env` for config is fine. What matters is that a deployed project genuinely runs end-to-end.

Demonstrate it works

Include at least a couple example projects that sync some source, for example Slack, Linear, or GitHub, etc. It should exercise non-trivial usage of the SDK and platform.

Stack

- TypeScript for the SDK(s): the authoring API and the client-side data API.
- A hosted API and server component
- Use Postgres as the underlying datastore. Should NOT use schema per tenant.
- Everything else (how you store schemas, run the code, schedule, design the query layer, libraries) is your call. Justify the important decisions.

What we're evaluating

- Correctness
- System design: how the SDKs, deployment, execution, storage, and query layer fit together for many tenants
- Code quality and judgment